



ALQUILA SENIOR

Plataforma de alquiler accesible
para personas mayores



Proyecto Intermodular

Desarrollo de Aplicaciones Multiplataforma (DAM)

Ricardo Sánchez de la Blanca Márquez

CESUR Andalucía

Curso 2025/2026

Índice general del documento

Resumen ejecutivo

1. Presentación general del proyecto
2. Objetivos, alcance y requisitos
3. Análisis funcional del sistema
4. Diseño técnico de la solución
5. Diseño de interfaz y accesibilidad
6. Planificación, recursos y ejecución
7. Gestión de riesgos, cambios e incidencias
8. Indicadores de calidad y evaluación del proyecto
9. Resultados esperados, validación y proyección futura
10. Conclusiones del proyecto
11. Consideraciones éticas, accesibilidad y protección de datos
12. Matrices y tablas de apoyo
13. Bibliografía y webgrafía
14. Conclusión de la memoria
15. Reflexión personal

Resumen ejecutivo

La presente memoria final integra y amplía las entregas previas del proyecto intermodular AlquilaSenior. El documento reúne la propuesta inicial, la identificación de la necesidad, el diseño técnico, la planificación del desarrollo y los apartados complementarios exigidos para la entrega definitiva: indicadores de calidad, gestión de cambios e incidencias, conclusiones y bibliografía. El resultado es una visión global del proyecto con formato académico, redacción unificada y recursos gráficos explicativos.

AlquilaSenior se plantea como una aplicación orientada a facilitar el acceso de personas mayores a viviendas y habitaciones adaptadas y asequibles. Su valor principal reside en la combinación de utilidad social y viabilidad técnica. A lo largo de la memoria se justifica la necesidad del proyecto, se describen sus objetivos, se detallan la arquitectura y el modelo de datos, se expone la planificación temporal y se establecen criterios para evaluar la calidad del resultado final.

1. Presentación general del proyecto

1.1. Contexto del proyecto

AlquilaSenior es una propuesta de aplicación multiplataforma orientada a facilitar el acceso de personas mayores a viviendas y habitaciones adaptadas, asequibles y fáciles de localizar. La idea parte de una necesidad social concreta: muchas personas de edad avanzada, especialmente aquellas que viven solas o cuentan con pensiones reducidas, encuentran dificultades para localizar una vivienda digna que responda a sus limitaciones económicas y de accesibilidad. El proyecto busca responder a ese problema mediante una solución técnica asumible dentro del marco académico del ciclo de Desarrollo de Aplicaciones Multiplataforma.

La elección del tema no solo tiene sentido desde el punto de vista técnico, sino también desde una perspectiva ética y social. En los últimos años el problema del acceso a la vivienda se ha acentuado, y en el caso de las personas mayores se suman obstáculos adicionales: dificultad para manejar portales inmobiliarios complejos, escasez de filtros centrados en accesibilidad y miedo a los trámites digitales. AlquilaSenior pretende simplificar ese proceso y reducir barreras de uso, enfocando la experiencia hacia un colectivo frecuentemente olvidado por las plataformas convencionales.

La solución se plantea desde un enfoque realista. En lugar de intentar desarrollar desde el inicio una aplicación con multitud de funciones, se ha optado por un producto mínimo viable que permite registrar usuarios, registrar viviendas, consultar inmuebles disponibles, aplicar filtros básicos y gestionar contratos de alquiler. Esta delimitación de alcance es

coherente con el tiempo disponible, con los conocimientos trabajados durante el ciclo y con la necesidad de garantizar una entrega completa, estable y bien documentada.

1.2. Problema detectado y oportunidad de mejora

El análisis previo de la necesidad mostró un desajuste claro entre la oferta y la demanda. Por un lado, existen propietarios con pisos o habitaciones disponibles que podrían alquilar a personas mayores responsables, tranquilas y estables. Por otro, hay usuarios que necesitan precisamente ese tipo de alojamiento, pero no encuentran un canal sencillo, específico y accesible para localizarlo. Las plataformas generalistas no suelen priorizar criterios como vivienda adaptada, facilidad de contacto, lenguaje claro o filtrado por capacidad económica real.

A ello se añade una dimensión práctica importante: muchas personas mayores dependen de familiares, vecinos o entidades sociales para buscar vivienda en internet. Esa dependencia reduce autonomía y puede retrasar decisiones críticas. Una aplicación pensada desde la claridad, con una interfaz directa y un proceso de búsqueda comprensible, puede convertirse en una herramienta de apoyo real. No se trata únicamente de digitalizar un trámite, sino de diseñar una experiencia que tenga en cuenta las limitaciones y expectativas del usuario final.

En este contexto, AlquilaSenior representa una oportunidad de mejora bien definida. La plataforma puede servir como puente entre propietarios y personas mayores, centralizando información relevante, simplificando la consulta de inmuebles y aplicando reglas de negocio que aporten orden al proceso. Aunque el desarrollo académico se limita a un MVP con interfaz por consola, la estructura diseñada deja preparada la evolución futura hacia interfaces gráficas o incluso una app móvil, tal y como reflejan los mockups conceptuales incluidos en la presente memoria.

1.3. Justificación académica y profesional

Desde el punto de vista académico, el proyecto integra de forma coherente competencias clave del ciclo DAM: análisis funcional, diseño técnico, modelado de datos, programación orientada a objetos, uso de patrones como MVC y DAO, persistencia en bases de datos relacionales, pruebas y documentación técnica. No se trata de un trabajo centrado en una única parte del temario, sino de una propuesta transversal que permite aplicar conocimientos adquiridos en varios módulos de manera coordinada.

Además, la temática elegida favorece un desarrollo profesional más maduro. Obliga a pensar en usuarios reales, reglas de negocio, restricciones del dominio y criterios de accesibilidad. También invita a justificar decisiones, priorizar funcionalidades y documentar

un plan de ejecución creíble. Estas habilidades son especialmente valiosas porque se acercan al tipo de razonamiento que se espera en un entorno profesional, donde no basta con programar, sino que es necesario comprender el problema que se está resolviendo.

Por último, la propuesta tiene valor personal para el alumno, ya que conecta la práctica técnica con una finalidad social concreta. Esta combinación refuerza la motivación durante el desarrollo, hace más significativo el aprendizaje y aporta coherencia al conjunto del proyecto. La memoria final recoge esa evolución: desde la detección del problema hasta la definición de una solución técnicamente viable, planificada y evaluable.

2. Objetivos, alcance y requisitos

2.1. Objetivo general

El objetivo general del proyecto es desarrollar una aplicación denominada AlquilaSenior que permita poner en contacto a personas mayores con propietarios de viviendas o habitaciones, facilitando la búsqueda de alojamientos adaptados y asequibles mediante una herramienta sencilla, clara y organizada. El sistema debe cubrir las operaciones esenciales del proceso: alta de usuarios, alta de inmuebles, consulta de viviendas disponibles y gestión básica de contratos.

2.2. Objetivos específicos

A partir del objetivo general, se concretan varios objetivos específicos. En primer lugar, modelar correctamente la información de personas mayores, propietarios, viviendas y contratos, estableciendo relaciones coherentes entre las entidades. En segundo lugar, implementar una capa de acceso a datos desacoplada de la lógica de negocio para facilitar el mantenimiento y la evolución del sistema. En tercer lugar, aplicar reglas funcionales críticas, como impedir que una vivienda tenga más de un contrato activo al mismo tiempo.

También se persigue diseñar una interfaz inicial lo suficientemente simple como para que el flujo principal pueda comprenderse sin esfuerzo. Aunque el MVP se resuelve por consola, el diseño debe ser compatible con futuras ampliaciones gráficas. Finalmente, un objetivo esencial es entregar una memoria completa y rigurosa, con criterios de calidad, planificación, gestión de cambios y conclusiones finales, de manera que el proyecto no solo funcione técnicamente, sino que esté correctamente justificado y documentado.

2.3. Alcance del MVP

El alcance funcional del producto mínimo viable se ha limitado de forma intencionada para asegurar la viabilidad temporal y la calidad de la solución. El MVP permite registrar personas mayores con datos básicos, registrar propietarios, registrar viviendas con precio mensual y nivel de adaptación, listar inmuebles, buscar viviendas según filtros simples y

generar contratos de alquiler. También contempla la finalización de contratos para mantener actualizada la disponibilidad de las viviendas.

No forman parte del alcance obligatorio de esta versión aspectos como el sistema de pagos, el envío automático de notificaciones, el registro web de usuarios, la autenticación avanzada, la geolocalización o la firma digital de documentos. Esas funcionalidades podrían ser valiosas en una versión profesional, pero introducirlas en esta fase aumentaría el riesgo de una implementación incompleta. La memoria reconoce esas posibilidades como líneas de evolución futura, pero mantiene el foco en un núcleo funcional claro y evaluable.

2.4. Requisitos funcionales y no funcionales

Los requisitos funcionales principales son los siguientes: registrar personas mayores, registrar propietarios, registrar viviendas, consultar viviendas disponibles, filtrar inmuebles por precio máximo y adaptación, crear contratos y finalizar contratos. Todos estos requisitos están alineados con la necesidad inicial detectada y pueden comprobarse mediante pruebas sencillas sobre el sistema. El cumplimiento de cada requisito se ha tenido en cuenta tanto en el diseño técnico como en la fase de planificación.

En cuanto a requisitos no funcionales, destacan la simplicidad de uso, la claridad del lenguaje empleado, la mantenibilidad del código, la separación de responsabilidades, la coherencia del modelo de datos y la posibilidad de ampliación futura. Además, se considera prioritario que la aplicación sea estable, legible y fácilmente depurable, dado que forma parte de un proyecto académico cuyo valor también depende de la calidad del proceso de desarrollo y de la claridad de la documentación asociada.

3. Análisis funcional del sistema

3.1. Actores implicados

El sistema se ha concebido para tres perfiles de interés. El primer perfil es el de la persona mayor, usuaria principal del servicio desde el punto de vista social. Aunque en el MVP la interacción puede estar mediada por un gestor o por el propio alumno durante la demostración, todas las decisiones de diseño parten de sus necesidades: claridad, accesibilidad y sencillez. El segundo perfil es el propietario, responsable de publicar y mantener la información de los inmuebles disponibles. El tercer perfil son las entidades sociales o personal de apoyo que, en una versión futura, podrían utilizar la herramienta para acompañar a personas mayores en el proceso de búsqueda.

Durante el desarrollo académico se ha simplificado la interacción considerando un gestor del sistema como actor operativo del menú. Esta decisión permite centrar la implementación en la lógica de negocio sin multiplicar complejidades innecesarias. No

obstante, la estructura de los datos y los flujos funcionales ya diferencia claramente entre personas mayores y propietarios, lo que permitirá evolucionar después hacia interfaces más personalizadas para cada rol.

3.2. Casos de uso principales

Los casos de uso más relevantes son registrar persona mayor, registrar propietario, registrar vivienda, buscar viviendas disponibles, crear contrato de alquiler y finalizar contrato. Cada uno de ellos cubre una necesidad concreta del proceso de intermediación entre oferta y demanda. Registrar a las personas y a los propietarios permite contar con datos mínimos estructurados. Registrar viviendas hace posible publicar la oferta. Buscar viviendas materializa el objetivo principal del sistema, y la gestión de contratos convierte la consulta en una operación con consecuencias reales sobre la disponibilidad del inmueble.

Estos casos de uso presentan una relación lógica entre sí. No puede crearse un contrato sin que existan previamente una persona mayor y una vivienda registrada. A su vez, la disponibilidad de una vivienda depende del estado de sus contratos. Esta dependencia entre operaciones justifica la existencia de una capa de servicio central, GestorAlquiler, capaz de validar precondiciones, coordinar DAOs y mantener la coherencia del sistema. El análisis funcional ha sido útil precisamente para identificar esas relaciones y evitar que la aplicación se limite a ser un simple conjunto de pantallas sin reglas internas.

3.3. Reglas de negocio

La regla de negocio más importante del sistema establece que una vivienda solo puede tener un contrato activo simultáneamente. Esta condición es esencial porque determina si un inmueble puede aparecer como disponible en el buscador y evita incoherencias graves en los datos. La existencia de varios contratos activos para la misma vivienda generaría resultados incorrectos, pérdida de confianza y errores en cascada. Por ello, la aplicación no debe limitarse a insertar registros, sino que ha de comprobar el estado previo antes de permitir nuevas altas de contrato.

Otras reglas relevantes son la necesidad de que toda vivienda esté asociada a un propietario válido, la conveniencia de validar importes y fechas con criterios coherentes y la obligación de mantener la integridad referencial entre tablas. También conviene revisar que la información de contacto tenga un formato razonable y que el sistema no ofrezca inmuebles ya vinculados a contratos activos. Aunque varias de estas validaciones podrían reforzarse más en una versión profesional, la memoria deja definido un marco funcional sólido y coherente para el desarrollo académico.

3.4. Flujo operativo principal

El flujo principal del sistema comienza con el registro de propietarios y personas mayores. Posteriormente se registran las viviendas disponibles, incluyendo dirección, importe y dato de adaptación. Cuando se desea buscar alojamiento, el sistema permite aplicar filtros básicos de consulta, especialmente por precio máximo y por adecuación a necesidades de accesibilidad. Tras revisar los resultados, puede formalizarse un contrato sobre una vivienda disponible.

En el momento de crear el contrato, la lógica de negocio comprueba si el inmueble ya tiene otro contrato activo. Si no lo tiene, se registra la operación y la vivienda deja de aparecer como disponible. Cuando el arrendamiento termina, se cierra el contrato mediante la fecha de finalización, lo que vuelve a habilitar el inmueble para búsquedas futuras. Este flujo, aunque sencillo, resume de manera eficaz el valor del proyecto y demuestra que el sistema responde a una necesidad concreta con operaciones perfectamente encadenadas.

4. Diseño técnico de la solución

4.1. Arquitectura software

El proyecto se apoya en una arquitectura en capas inspirada en el patrón MVC y reforzada mediante el uso del patrón DAO para el acceso a datos. Esta elección responde a un criterio fundamental de calidad: separar responsabilidades. La capa de presentación se limita a mostrar menús, recoger datos y presentar resultados. La lógica de negocio, centralizada en la clase *GestorAlquiler*, decide qué operaciones son válidas y en qué orden deben ejecutarse. El modelo representa las entidades del dominio, mientras que los DAOs encapsulan las consultas a la base de datos.

Esta estructura ofrece varias ventajas relevantes. En primer lugar, hace que el código sea más legible y mantenible, ya que cada clase tiene un propósito claro. En segundo lugar, facilita las pruebas, porque la lógica del negocio puede revisarse con independencia de la interfaz. En tercer lugar, prepara la aplicación para futuras ampliaciones, como una interfaz gráfica en JavaFX o una API REST. Por ello, la arquitectura propuesta no solo resuelve la necesidad actual, sino que deja una base técnica sólida para la evolución del sistema.

4.2. Modelo de dominio

El modelo de dominio se compone de cuatro entidades principales: *PersonaMayor*, *Propietario*, *Vivienda* y *Contrato*. *PersonaMayor* almacena la información básica del usuario que busca alojamiento. *Propietario* representa a quien pone un inmueble en alquiler. *Vivienda* contiene los datos del recurso habitacional, incluidos el precio mensual y el

indicador de adaptación. Contrato, por su parte, refleja la formalización de la relación entre una persona mayor y una vivienda durante un periodo determinado.

Cada entidad cuenta con atributos privados y métodos de acceso, siguiendo principios básicos de encapsulación. Aunque se trata de un modelo deliberadamente sencillo, resulta suficiente para expresar el núcleo del problema de negocio. Además, la claridad de este diseño favorece tanto la implementación en Java como la comprensión de la memoria técnica. La estructura puede crecer en el futuro incorporando validaciones más ricas, comportamiento propio en las entidades o nuevos objetos de valor, pero la base definida es coherente con el alcance del proyecto.

4.3. Persistencia y patrón DAO

El patrón DAO se ha elegido para aislar las operaciones de persistencia del resto del sistema. En lugar de que la interfaz o la lógica de negocio ejecuten consultas SQL directamente, cada entidad dispone de una clase DAO responsable de operaciones como crear registros, buscar por identificador, listar elementos o consultar inmuebles disponibles. Este desacoplamiento reduce dependencia entre capas y permite modificar la tecnología de almacenamiento con un impacto mucho menor sobre la aplicación.

La memoria contempla como opciones de persistencia tanto SQLite como MySQL. SQLite resulta especialmente adecuada para un entorno local o académico, ya que no requiere una infraestructura compleja y permite trabajar de forma rápida sobre un archivo de base de datos. MySQL, en cambio, se plantea como alternativa para un entorno más próximo a un despliegue cliente-servidor. La elección entre ambos motores no afecta a la arquitectura general gracias a la intermediación de los DAOs y de la clase de conexión a base de datos.

4.4. Diseño de la base de datos

El modelo relacional propuesto responde bien a la naturaleza de la información gestionada. Las relaciones entre personas, propietarios, viviendas y contratos son estables, bien delimitadas y susceptibles de expresarse mediante claves primarias y foráneas. La tabla PERSONA_MAYOR recoge datos básicos del usuario final; PROPIETARIO almacena la identidad del arrendador; VIVIENDA se vincula al propietario mediante una clave foránea; y CONTRATO enlaza a la persona mayor con la vivienda durante un intervalo temporal determinado.

El diseño incorpora una restricción funcional muy importante: aunque históricamente una vivienda puede aparecer en varios contratos, en un momento dado solo puede tener uno activo. Esta decisión no depende únicamente del esquema físico, sino de una regla de

negocio validada por el servicio. En conjunto, la base de datos propuesta garantiza integridad, claridad y una buena correspondencia entre el mundo real y la representación digital del sistema.

4.5. Escalabilidad y mejoras futuras

Aunque el desarrollo actual se ajusta a un MVP, la solución ha sido diseñada con visión de crecimiento. La primera línea de evolución natural sería sustituir o complementar la interfaz por consola por una interfaz gráfica accesible, por ejemplo, con JavaFX, o por una app móvil conectada a un backend común. También podría incorporarse autenticación, perfiles diferenciados, carga de documentación, notificaciones y paneles de seguimiento para entidades sociales.

A nivel técnico, sería razonable introducir una capa de servicios más amplia, validaciones específicas por entidad, pruebas de integración automatizadas y una API que expusiera operaciones del sistema a clientes externos. Del mismo modo, en un contexto profesional podrían revisarse aspectos como el cifrado de datos sensibles, la gestión de permisos y la trazabilidad de cambios. La memoria final no presenta estas mejoras como promesas vacías, sino como una hoja de ruta plausible construida sobre la arquitectura ya definida.

Figura 1. Arquitectura por capas de AlquilaSenior

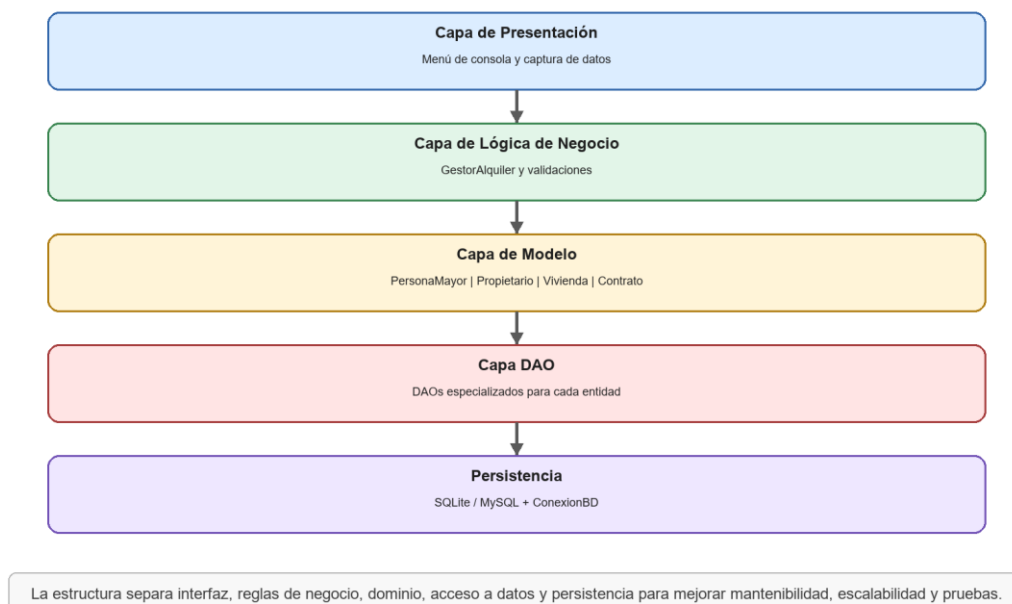
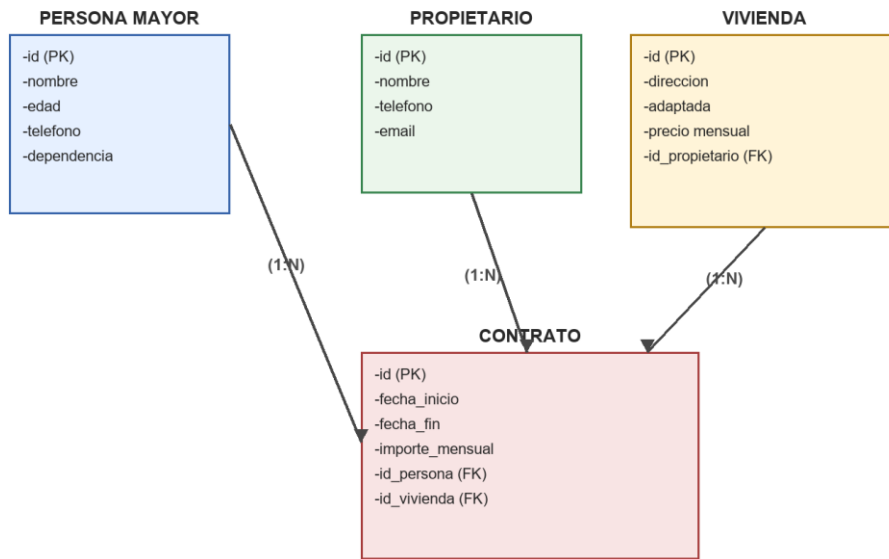


Figura 1. Arquitectura por capas empleada en AlquilaSenior.

DIAGRAMA ENTIDAD-RELACION - ALQUILASENIOR



Aunque la relacion entre Vivienda y Contrato es 1:N, el sistema impone como regla de negocio que solo pueda existir un contrato activo por vivienda en un momento dado.

Figura 2. Modelo entidad-relación resumido de la base de datos.

DIAGRAMA UML ALQUILASENIOR

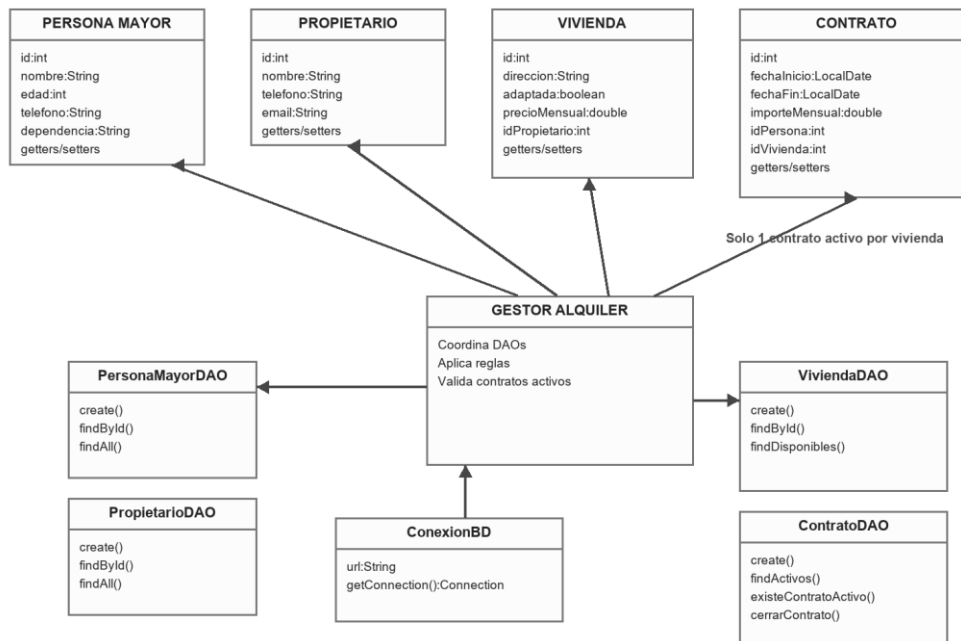


Figura 3. Diagrama UML simplificado de entidades, DAOs y servicio.

5. Diseño de interfaz y accesibilidad

5.1. Interfaz inicial del MVP

La primera versión de AlquilaSenior se apoya en una interfaz por consola. Esta decisión responde a criterios de viabilidad: permite dedicar el esfuerzo principal al diseño del dominio, a la persistencia y a la lógica de negocio, sin dispersar recursos en una interfaz compleja. A pesar de su sencillez, la interfaz se ha pensado con un enfoque claro: menús numerados, mensajes comprensibles, confirmaciones explícitas y reducción de pasos innecesarios. En un proyecto académico, esta aproximación ayuda a demostrar la funcionalidad real del sistema sin comprometer su estabilidad.

Lejos de ser una limitación absoluta, esta elección actúa como un primer escalón. El diseño técnico separa lo suficiente la presentación del resto de capas como para que una futura interfaz gráfica pueda sustituir a la consola sin reescribir toda la lógica de negocio. Esta capacidad de evolución es uno de los indicadores de calidad del proyecto, porque muestra que la implementación actual no se ha construido de forma improvisada, sino sobre una base estructurada.

5.2. Propuesta visual futura

Los mockups incluidos en el material del proyecto permiten visualizar una posible evolución hacia una aplicación móvil o una interfaz gráfica más intuitiva. En esas pantallas se observan decisiones muy concretas: botones amplios, jerarquía visual clara, textos breves, acciones principales destacadas y flujos secuenciales. La navegación planteada evita saturar al usuario con información y prioriza la consulta de viviendas y el acceso a los detalles más relevantes para la toma de decisiones.

Esta propuesta visual tiene sentido porque el público objetivo no siempre está habituado a interactuar con herramientas digitales complejas. Diseñar para personas mayores exige renunciar a la sobrecarga innecesaria y centrar la experiencia en la comprensión inmediata. Por eso, la memoria considera los mockups no como un elemento decorativo, sino como un recurso de análisis de usabilidad y como evidencia de que la futura evolución del sistema ya ha sido pensada desde la accesibilidad.

5.3. Criterios de accesibilidad aplicados

Entre los criterios de accesibilidad considerados destacan el uso de un lenguaje sencillo, la reducción del número de acciones por pantalla, la visibilidad de la información

crítica y la consistencia del flujo de navegación. En una futura versión gráfica, estos criterios deberían traducirse además en tamaños de fuente generosos, buen contraste, iconografía clara y formularios breves. La plataforma debe ayudar a tomar decisiones, no convertirse en una barrera por sí misma.

También se ha tenido en cuenta la importancia del contexto emocional del usuario. Buscar vivienda puede ser una experiencia estresante, especialmente cuando intervienen limitaciones económicas o de salud. Por ello, una solución bien diseñada debe transmitir orden, seguridad y predictibilidad. Aunque el proyecto no entra todavía en estudios de usabilidad formales, la presente memoria muestra una orientación de diseño coherente con las necesidades reales del colectivo al que se dirige.

ALQUILASENIOR MOCKUPS MOVIL



Se presenta un conjunto de mockups conceptuales de la interfaz móvil de la aplicación AlquilaSenior. Estos prototipos representan una posible evolución futura del sistema hacia una aplicación multiplataforma.

MOCKUPS MOVIL



RESULTADOS	DETALLES VIVIENDA
 Calle Mayor 12  450 €/mes  Adaptada: Sí	 Dirección: Calle Mayor 12
 Av. Andalucía 5  380 €/mes  Adaptada: No	 Precio mensual: 450 €/mes
	 Vivienda adaptada: Sí
	 Propietario: Contacto disponible
VER DETALLES	SOLICITAR ALQUILER

Se presenta un conjunto de mockups conceptuales de la interfaz móvil de la aplicación AlquilaSenior. Estos prototipos representan una posible evolución futura del sistema hacia una aplicación móvil multiplataforma. El diseño se ha realizado priorizando la simplicidad, la accesibilidad y la claridad visual, aspectos fundamentales para personas mayores. Las pantallas reflejan el flujo principal de la aplicación: acceso al menú, búsqueda de viviendas, visualización de resultados y consulta detallada de una vivienda para la solicitud de alquiler.

Figura 4. Propuesta de evolución hacia interfaz móvil accesible.

6. Planificación, recursos y ejecución

6.1. Enfoque de desarrollo

La planificación del proyecto se ha organizado siguiendo un enfoque incremental. Primero se realizó la identificación de la necesidad y la delimitación del problema. Después se elaboró el diseño técnico, incluyendo arquitectura, base de datos, UML y recursos visuales. En una tercera fase se aborda el desarrollo de clases del dominio, acceso a datos y lógica de negocio. Posteriormente se reservó tiempo para pruebas, corrección de errores y documentación final. Esta secuencia permite reducir riesgos y construir sobre bases bien definidas.

El enfoque incremental es especialmente útil en un proyecto académico individual porque evita iniciar la implementación sin una guía clara. Además, facilita la revisión del trabajo por entregas, permite justificar mejor las decisiones y ayuda a detectar carencias antes de que sea demasiado tarde para corregirlas. En este caso, cada fase produce un resultado tangible: análisis, diseño, planificación, materiales gráficos o memoria técnica.

6.2. Cronograma y secuencia de tareas

El diagrama de Gantt incluido en la memoria no debe interpretarse como una simple imagen orientativa, sino como una representación resumida de la lógica temporal del proyecto. La secuencia parte del análisis del problema y de la definición de requisitos, porque sin esa base no tendría sentido diseñar el modelo de datos ni la arquitectura. A continuación, se sitúan el diseño funcional y la configuración del entorno, que preparan el terreno para desarrollar las clases del dominio, la capa DAO y la lógica de negocio. Solo cuando esas piezas están coordinadas resulta razonable reservar tiempo para pruebas, correcciones y documentación final.

La dependencia principal del cronograma es, por tanto, acumulativa: cada bloque necesita resultados del anterior. El modelado del dominio depende del análisis; la persistencia depende del modelo relacional; GestorAlquiler depende de las entidades y DAOs; y la validación depende de que el flujo principal ya exista. Esta lógica temporal es importante porque justifica el orden de trabajo y demuestra que la planificación no se ha construido como una lista de fases aisladas, sino como una cadena de tareas relacionadas entre sí.

6.3. Recursos humanos y técnicos

El desarrollo se plantea como un proyecto individual, con un único desarrollador responsable del análisis, diseño, implementación, pruebas y documentación. Esta condición obliga a mantener un alcance realista y a priorizar las tareas esenciales. Aun así, también aporta coherencia al proyecto, porque todas las decisiones responden a una misma línea de trabajo y a un mismo criterio de diseño.

En cuanto a recursos técnicos, la propuesta contempla un ordenador personal, un entorno de desarrollo como Eclipse o equivalente, Java 17, una base de datos SQLite o MySQL y librerías de prueba como JUnit. Son herramientas estables, conocidas durante el ciclo y adecuadas para el nivel de complejidad del proyecto. La elección de tecnologías gratuitas y ampliamente documentadas también contribuye a la viabilidad económica y técnica del conjunto.

6.4. Presupuesto estimado

Aunque se trata de un proyecto académico, resulta útil realizar una estimación económica para adoptar una perspectiva más profesional. La documentación previa sitúa el desarrollo en unas ochenta horas valoradas a quince euros por hora, lo que produce un

coste directo de 1.200 euros. A ello se añade una estimación de 200 euros en recursos técnicos y herramientas, alcanzando un total aproximado de 1.400 euros.

Este presupuesto no debe interpretarse como un coste comercial cerrado, sino como una referencia formativa. Su valor radica en acostumbrar al alumno a pensar en tiempo, esfuerzo y recursos. También permite entender que la toma de decisiones técnicas tiene impacto económico: una arquitectura ordenada, una buena planificación y la reducción de errores no solo mejoran la calidad, sino que también disminuyen costes de mantenimiento y retrabajo.

2. CRONOGRAMA DEL PROYECTO

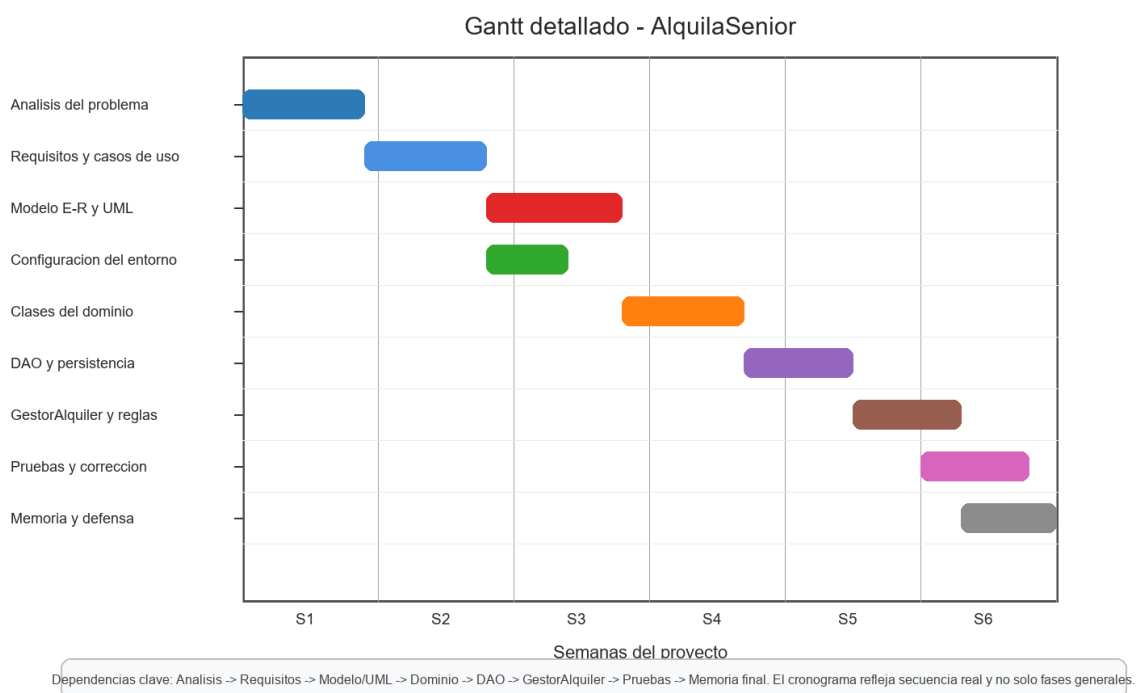


Figura 5. Cronograma general del proyecto.

7. Gestión de riesgos, cambios e incidencias

7.1. Riesgos identificados

Entre los riesgos principales del proyecto destacan los errores en la lógica de negocio, los retrasos en el calendario, los problemas de persistencia y las incidencias derivadas de la integración entre capas. Un error en la regla de disponibilidad de viviendas podría invalidar el sentido completo del sistema. Del mismo modo, una mala gestión de fechas o de claves

foráneas puede provocar inconsistencias difíciles de detectar. Por eso, la identificación temprana de riesgos forma parte de la calidad del proyecto y no debe considerarse un apartado meramente formal.

Para valorar estos riesgos de un modo más profesional se utilizan tres variables: probabilidad de ocurrencia, impacto sobre la entrega y capacidad de respuesta. No todos los riesgos son igual de peligrosos. Por ejemplo, un pequeño ajuste visual en la documentación tiene impacto bajo, mientras que un fallo en contratos activos o un retraso acumulado en pruebas tiene impacto alto. Esta distinción ayuda a priorizar esfuerzos y a decidir en qué puntos conviene actuar de forma preventiva antes de que aparezca la incidencia.

7.2. Medidas de mitigación

Para reducir el impacto de los riesgos detectados se proponen varias medidas. La más importante es centralizar las reglas de negocio en la clase GestorAlquiler, evitando que la interfaz o los DAOs actúen por su cuenta en decisiones críticas. A ello se suman validaciones de entrada, control del estado de los contratos, comprobación de relaciones entre entidades y pruebas unitarias sobre operaciones sensibles. Estas medidas no eliminan por completo la posibilidad de error, pero sí reducen notablemente su probabilidad y facilitan su detección.

Desde el punto de vista organizativo, la principal estrategia de mitigación es la planificación por fases con hitos claros y con margen de contingencia. Cada entrega previa ha servido como mecanismo de control y revisión, permitiendo detectar aspectos mejorables antes de la redacción de la memoria final. Si una tarea crítica se retrasa, el plan de contingencia consiste en congelar el alcance no esencial, reforzar las pruebas del núcleo funcional y priorizar estabilidad sobre nuevas funcionalidades. Esta respuesta es coherente con el enfoque MVP y aumenta la probabilidad de una entrega completa y defendible.

7.3. Gestión de cambios

La evolución del proyecto entre entregas muestra una gestión de cambios razonable. La idea inicial ya definía el propósito social de la aplicación, pero en fases posteriores se concretó mejor la arquitectura, se reforzó la centralización de la lógica de negocio y se introdujo una visión más clara de escalabilidad. Estos cambios no alteran la esencia del proyecto; al contrario, la consolidan y la convierten en una propuesta técnicamente más coherente.

La gestión de cambios se ha orientado por tres criterios: mantener la necesidad original, mejorar la precisión técnica y no aumentar artificialmente el alcance. Por ejemplo, la incorporación de GestorAlquiler como núcleo del sistema mejora la claridad arquitectónica sin obligar a rediseñar todo el proyecto. Del mismo modo, los mockups no cambian el MVP actual, pero aportan una visión futura de la experiencia de usuario. Esta forma de introducir cambios es adecuada porque aporta valor sin comprometer la viabilidad.

7.4. Gestión de incidencias

En un proyecto de estas características, una incidencia puede definirse como cualquier problema que impide avanzar o que genera un comportamiento distinto al esperado. Para gestionarlas de forma ordenada conviene clasificarlas según su gravedad, registrar el contexto en el que aparecen, anotar la causa probable y documentar la solución aplicada. Aunque el trabajo académico no requiere un sistema formal de tickets, adoptar esta mentalidad ayuda a profesionalizar el desarrollo.

En AlquilaSenior, las incidencias más previsibles están relacionadas con datos inconsistentes, consultas SQL incorrectas, errores de validación y comportamientos no contemplados en el menú. La memoria final propone tratarlas con un criterio sencillo: detectar, reproducir, corregir, verificar y documentar. Este ciclo permite convertir errores puntuales en aprendizaje estructurado y contribuye a que el proyecto resulte más sólido y defendible.

8. Indicadores de calidad y evaluación del proyecto

8.1. Criterios de calidad definidos

La calidad del proyecto no depende únicamente de que el programa compile o de que la base de datos funcione. En esta memoria se entiende la calidad como la combinación de varios factores: adecuación a la necesidad detectada, cobertura de requisitos funcionales, claridad arquitectónica, coherencia del modelo de datos, facilidad de mantenimiento, accesibilidad del diseño y nivel de formalidad de la documentación. Esta visión amplia es importante porque el proyecto intermodular debe demostrar tanto competencias técnicas como capacidad de análisis y presentación profesional.

Bajo ese enfoque, se han establecido indicadores concretos que permitan valorar el resultado. Algunos son funcionales, como la cobertura del MVP o la correcta aplicación de la regla de un contrato activo por vivienda. Otros son estructurales, como la separación entre presentación, lógica y persistencia. También hay indicadores documentales, por ejemplo, la presencia de índice, figuras numeradas, bibliografía, planificación y conclusiones

argumentadas. La combinación de estos indicadores permite una evaluación más objetiva del trabajo realizado.

8.2. Método de evaluación

La evaluación del proyecto puede plantearse en tres niveles. En primer lugar, una comprobación funcional del sistema, revisando si cada caso de uso principal puede ejecutarse correctamente y con resultados coherentes. En segundo lugar, una revisión técnica del diseño, valorando si la arquitectura elegida está bien justificada y si el modelo de datos responde al problema identificado. En tercer lugar, una evaluación documental que tenga en cuenta la claridad de la memoria, la calidad de las figuras y el grado de cumplimiento de las normas de entrega indicadas por el profesor.

Este método de evaluación es especialmente adecuado para un proyecto académico porque evita reducir la nota a una única dimensión. Un sistema puede estar bien planteado, pero mal documentado, o presentar una memoria cuidada pero una arquitectura débil. La propuesta de evaluación recogida en esta memoria busca precisamente equilibrar resultado funcional, rigor técnico y calidad expositiva. Así, la valoración final refleja de manera más fiel el trabajo intermodular realizado.

8.3. Evidencias de cumplimiento

Las principales evidencias de cumplimiento son las entregas previas, los diagramas incluidos, el cronograma, la estimación de recursos, la definición de riesgos y la presente memoria final integradora. La existencia de materiales como el diagrama entidad-relación, el UML y los mockups confirma que el proyecto no se ha limitado a una descripción abstracta, sino que ha evolucionado hacia una solución estructurada y visualmente explicable.

Además, la incorporación de apartados adicionales exigidos por el profesor, como indicadores de calidad, gestión de cambios e incidencias, conclusiones y bibliografía, demuestra capacidad de adaptación a los criterios de evaluación. Esta memoria no sustituye a las entregas anteriores, sino que las reúne, amplía y articula en un documento único más maduro y profesional, lo que constituye por sí mismo una evidencia de progreso.

9. Resultados esperados, validación y proyección futura

9.1. Resultados esperados del MVP

El resultado esperado del producto mínimo viable es una aplicación funcional capaz de cubrir el circuito básico del problema planteado. Eso significa que debe permitir registrar información coherente, recuperar datos almacenados, identificar inmuebles disponibles, filtrar resultados relevantes y controlar correctamente la ocupación de las viviendas

mediante contratos. Si estas operaciones se ejecutan con estabilidad y claridad, el proyecto habrá cumplido el núcleo de su propósito académico y funcional.

Igualmente, se espera que el proyecto sirva como demostración del dominio de conceptos trabajados en el ciclo. La memoria, la arquitectura, el modelo relacional y la organización por capas deben evidenciar que el desarrollo no se ha realizado de forma improvisada. En otras palabras, el resultado esperado no es solo un programa, sino una solución razonada, justificable y defendible.

9.2. Estrategia de validación

La validación del sistema puede realizarse mediante pruebas funcionales centradas en escenarios típicos: alta correcta de personas mayores, alta de viviendas vinculadas a propietarios, búsqueda con filtros, creación válida de un contrato, rechazo de un contrato duplicado y cierre de un contrato existente. Estos casos de prueba permiten comprobar que los elementos principales del negocio están correctamente coordinados.

Junto a las pruebas funcionales, conviene revisar también la consistencia de la interfaz y la claridad del flujo. Incluso en consola, el sistema debe guiar al usuario sin ambigüedad, responder con mensajes comprensibles y ofrecer confirmaciones adecuadas. La validación, por tanto, no se limita a los datos, sino que incluye la experiencia de uso y la coherencia global del producto.

9.3. Proyección futura del proyecto

La proyección futura de AlquilaSenior es uno de los aspectos con mayor potencial. Una vez consolidado el núcleo funcional, la evolución más inmediata sería disponer de una interfaz gráfica completa y accesible, manteniendo la misma capa de negocio. Posteriormente podría desarrollarse una aplicación móvil multiplataforma o una solución web con backend API, de modo que propietarios y personas mayores interactúen directamente con el servicio.

También serían valiosas mejoras como la incorporación de filtros geográficos, historial de solicitudes, notificaciones, perfiles verificados, evaluación de inmuebles, soporte documental o integración con entidades sociales. Sin embargo, la memoria insiste en que el mérito del proyecto no reside en prometer todas estas funciones, sino en haber creado una base coherente a partir de la cual esas ampliaciones resultan técnicamente plausibles.

10. Conclusiones finales del proyecto

10.1. Valoración global

La elaboración de AlquilaSenior ha permitido transformar una necesidad social concreta en una propuesta técnica clara, viable y alineada con los aprendizajes del ciclo DAM. El proyecto destaca por su coherencia interna: parte de un problema real, define un alcance asumible, propone una arquitectura adecuada, organiza la información mediante un modelo relacional y planifica su desarrollo con criterios razonables. Esa coherencia es, en sí misma, uno de los principales logros del trabajo.

Más allá del componente técnico, el proyecto aporta valor por su orientación a un colectivo con necesidades específicas. En un contexto donde muchas soluciones digitales se diseñan pensando en usuarios avanzados, AlquilaSenior propone una mirada distinta: simplificar, hacer accesible y enfocar el desarrollo hacia una utilidad social tangible. Esta perspectiva enriquece el proyecto y le aporta una identidad clara.

10.2. Aprendizajes adquiridos

Durante el proceso de elaboración se consolidan aprendizajes relacionados con análisis de requisitos, modelado del dominio, bases de datos relacionales, diseño por capas, aplicación de patrones y redacción técnica. También se refuerza la idea de que programar no consiste únicamente en escribir código, sino en tomar decisiones justificadas, priorizar, validar hipótesis y documentar correctamente el camino seguido.

Otro aprendizaje importante es la necesidad de mantener el equilibrio entre ambición y viabilidad. Un proyecto excesivamente grande corre el riesgo de quedarse a medias; uno demasiado simple puede resultar pobre desde el punto de vista académico. La evolución de AlquilaSenior muestra una búsqueda consciente de ese equilibrio, que se ha concretado en un MVP sólido y en una proyección futura realista.

10.3. Cierre

En conclusión, la memoria final demuestra que el proyecto ha sido desarrollado con criterio, con una orientación clara al usuario y con una estructura técnica consistente. Las entregas previas han servido como base para construir un documento único más maduro, mejor organizado y ajustado a los requisitos formales de la evaluación. El resultado final ofrece una visión completa del proyecto, desde la necesidad detectada hasta los criterios de calidad y las posibilidades de crecimiento.

Por todo ello, AlquilaSenior puede considerarse una propuesta académica completa, útil y bien fundamentada. Refleja no solo conocimientos técnicos, sino también capacidad de

análisis, planificación y sensibilidad hacia el problema que intenta resolver. Esa combinación es la que convierte el trabajo en una entrega sólida y aspirante a la máxima calificación.

11. Consideraciones éticas, accesibilidad y protección de datos

11.1. Dimensión ética del proyecto

AlquilaSenior no debe entenderse únicamente como un ejercicio técnico, sino como una propuesta que interactúa con una realidad social sensible. El público al que se dirige, formado principalmente por personas mayores, puede encontrarse en situaciones de vulnerabilidad económica, dependencia parcial o baja alfabetización digital. Por ello, cualquier solución orientada a este colectivo debe construirse desde una lógica de respeto, claridad y utilidad real. Diseñar para personas mayores no consiste en simplificar por paternalismo, sino en eliminar barreras innecesarias y facilitar que el usuario tome decisiones con autonomía y comprensión.

Desde una perspectiva ética, el proyecto evita enfoques discriminatorios y se orienta a la igualdad de acceso. La plataforma no pretende etiquetar a las personas por su edad, sino responder a necesidades concretas que las plataformas generalistas atienden de forma insuficiente. En este sentido, el propio planteamiento del sistema puede interpretarse como una medida de inclusión digital. La tecnología se pone al servicio del usuario, en lugar de exigir que el usuario se adapte a una interfaz compleja o a procesos diseñados sin tener en cuenta su contexto.

También es importante señalar que la relación entre necesidad social y tecnología obliga a ser prudentes en la formulación de expectativas. Una herramienta como AlquilaSenior puede mejorar el acceso a información y ordenar el proceso de búsqueda, pero no resuelve por sí sola el problema estructural de la vivienda. Reconocer ese límite es una actitud honesta y profesional. El proyecto aporta una solución parcial, concreta y viable a una parte del problema: la intermediación sencilla y orientada a un perfil específico.

11.2. Protección de datos y privacidad

El sistema maneja datos personales básicos de personas mayores y propietarios, por lo que la privacidad es un aspecto que no debe ignorarse, aunque el proyecto tenga carácter académico. Nombres, teléfonos, direcciones de vivienda y posibles referencias a niveles de dependencia son informaciones sensibles que requieren un tratamiento responsable. En una versión profesional del producto sería imprescindible aplicar principios alineados con la normativa de protección de datos, como la minimización de información, el acceso

restringido según perfil y la conservación limitada de los datos estrictamente necesarios para prestar el servicio.

En el diseño actual ya existen decisiones que van en esa dirección, aunque de forma preliminar. El modelo de datos se mantiene sencillo y solo recoge los campos que resultan útiles para la lógica principal del sistema. No se almacenan, por ejemplo, datos médicos extensos ni información financiera avanzada. Esta contención es positiva porque reduce riesgo y facilita la comprensión del modelo. Aun así, una evolución real del proyecto debería contemplar cifrado de información sensible, control de usuarios, registro de accesos y mecanismos claros para actualizar o eliminar datos cuando corresponda.

La memoria considera especialmente importante el dato relativo a la dependencia, ya que puede influir en los criterios de accesibilidad de la vivienda y, al mismo tiempo, afectar a la privacidad de la persona mayor. Por eso conviene tratarlo con prudencia, almacenarlo de manera estructurada y utilizarlo solo para mejorar la adecuación del servicio. En un escenario real, el consentimiento informado y la transparencia sobre la finalidad del tratamiento serían indispensables. Incluir esta reflexión refuerza la madurez del proyecto y muestra sensibilidad hacia un aspecto clave del desarrollo de software actual.

11.3. Accesibilidad como criterio de calidad

La accesibilidad no debe concebirse como un añadido opcional, sino como un criterio de calidad central en un proyecto con este enfoque. De hecho, parte del valor de AlquilaSenior reside en reconocer que la barrera no se encuentra solo en la falta de viviendas adecuadas, sino también en la dificultad para localizar y comprender la información disponible. Si la herramienta reproduce la complejidad de los portales generalistas, dejaría de cumplir su función diferencial. Por ello, el proyecto ha incorporado desde etapas tempranas una atención específica al diseño simple, al lenguaje claro y a la lógica de navegación ordenada.

En el MVP por consola, la accesibilidad se traduce en menús directos, textos comprensibles y una secuencia previsible de acciones. En la proyección futura hacia interfaz gráfica o móvil, debería concretarse además en tipografías legibles, contraste suficiente, botones amplios, iconografía reconocible y reducción de sobrecarga visual. Este planteamiento está ya reflejado en los mockups conceptuales incluidos como figura en la memoria. No se trata solo de que la interfaz sea bonita, sino de que acompañe a usuarios con necesidades concretas sin generar frustración ni dependencia innecesaria de terceros.

La accesibilidad también implica pensar en la carga cognitiva del usuario. Cuantos menos pasos, menos jerga técnica y menos decisiones simultáneas se exijan, mayor será la probabilidad de éxito de la interacción. Esta reflexión aporta un enfoque más profesional al

proyecto, porque demuestra que el diseño se ha entendido como parte de la solución y no como una capa estética añadida al final del desarrollo.

11.4. Sostenibilidad y utilidad social

Aunque el término sostenibilidad suele asociarse a aspectos medioambientales, en proyectos de software con dimensión social también puede interpretarse en clave de utilidad, mantenimiento y continuidad. Un sistema sostenible es aquel que resuelve una necesidad real sin depender de una complejidad desproporcionada. En ese sentido, AlquilaSenior busca ser sostenible porque se apoya en tecnologías accesibles, de bajo coste y bien conocidas, evitando dependencias sofisticadas que dificulten su evolución. La sencillez técnica favorece que el proyecto pueda mantenerse, explicarse y ampliarse de forma gradual.

Además, la utilidad social del proyecto aporta una capa adicional de sostenibilidad. Una herramienta que facilita el acceso a vivienda para personas mayores y que puede llegar a ser utilizada por entidades sociales o servicios de apoyo tiene un impacto potencial mayor que el de una mera práctica académica sin aplicación contextual. El proyecto demuestra que el desarrollo de software puede orientarse a problemas cotidianos y a colectivos concretos, generando valor más allá del aula. Esta idea resulta especialmente relevante en la formación profesional, donde la conexión entre aprendizaje y realidad social debería ocupar un lugar central.

Por todo ello, incorporar consideraciones éticas, de privacidad, accesibilidad y sostenibilidad no es un adorno teórico, sino una forma de reforzar la calidad integral de la propuesta. La memoria final gana profundidad al incluir estos aspectos y muestra una comprensión más completa de lo que significa diseñar una aplicación útil, responsable y preparada para crecer.

12. Matrices y tablas de apoyo

12.1. Matriz resumida de requisitos

Requisito	Descripción	Prioridad	Criterio de verificación
RF-01	Registrar personas mayores	Alta	Alta correcta y recuperación posterior
RF-02	Registrar propietarios	Alta	Persistencia de datos y relación con viviendas
RF-03	Registrar viviendas	Alta	Asociación válida a propietario
RF-04	Buscar viviendas disponibles	Alta	Listado filtrado por

			precio y adaptación
RF-05	Crear contratos	Alta	Alta solo si la vivienda no está ocupada
RF-06	Finalizar contratos	Media	Cambio de estado y liberación de vivienda
RNF-01	Arquitectura mantenible	Alta	Separación en capas MVC + DAO
RNF-02	Interfaz comprensible	Alta	Mensajes claros y flujo sencillo

12.2. Presupuesto resumido

Concepto	Estimación
Desarrollo del software (80 h x 15 €/h)	1.200 €
Recursos técnicos y herramientas	200 €
Coste total estimado	1.400 €

12.3. Cronograma detallado y dependencias

Tarea	Duración	Dependencia principal	Resultado esperado
Análisis del problema y requisitos	1 semana	Inicio del proyecto	Necesidad delimitada y requisitos base
Diseño funcional y casos de uso	1 semana	Análisis aprobado	Flujo principal definido
Modelo de datos y diagrama E-R	1 semana	Requisitos y casos de uso	Estructura relacional coherente
Configuración del entorno	3 días	Diseño técnico inicial	Proyecto preparado para desarrollo
Clases del dominio	1 semana	Modelo de datos	Entidades implementadas
DAO y persistencia	1 semana	Clases del dominio y base de datos	Acceso estable a datos
GestorAlquiler y reglas de negocio	1 semana	Dominio y DAO	Lógica central operativa
Pruebas funcionales y corrección	4 días	Flujo principal implementado	Errores críticos reducidos
Memoria final y defensa	4 días	Versión validada	Entrega completa y presentable

El cronograma desglosado muestra que las tareas no solo se suceden por bloques generales, sino que mantienen dependencias directas. Por ejemplo, GestorAlquiler no puede cerrarse antes de disponer de entidades y persistencia, y las pruebas no tienen sentido hasta contar con el flujo principal operativo.

Este detalle permite justificar mejor la secuenciación real del trabajo y muestra una planificación más cercana a un entorno profesional, aunque adaptada al alcance académico del proyecto.

12.4. Matriz de riesgos ampliada

Riesgo	Probabilidad	Impacto	Prevención	Contingencia
Error en contratos activos	Media	Alto	Validación central en GestorAlquiler y casos de prueba	Bloquear altas conflictivas y revisar trazas del contrato
Retrasos por alcance excesivo	Media	Alto	Priorización estricta del MVP y control semanal	Congelar funcionalidades no esenciales y reforzar el núcleo
Problemas de persistencia	Media	Media	Uso de DAO, consultas revisadas y pruebas sobre BD	Corregir capa de acceso y repetir pruebas de integración
Inconsistencias de datos	Baja	Media	Integridad relacional, validaciones y datos de prueba	Depurar registros afectados y rehacer inserciones críticas
Documentación final insuficiente	Baja	Media	Avance parcial de memoria en paralelo al desarrollo	Reasignar tiempo final y simplificar partes no esenciales

12.5. Indicadores de calidad y evaluación

Figura 6. Cuadro de indicadores de calidad y evaluación

Indicador	Meta	Evidencia de evaluación
Cobertura funcional del MVP	>= 90 %	Verificación manual de requisitos y casos de uso
Validez de reglas de negocio	100 % reglas críticas	Pruebas sobre contratos activos y disponibilidad
Claridad de la arquitectura	Alta	Separación en capas MVC + DAO
Accesibilidad de la interfaz	Cumple criterios definidos	Revisión de lenguaje, contraste y flujo
Calidad documental	Entrega completa	Memoria, figuras, índice y bibliografía

Figura 6. Resumen visual de indicadores de calidad.

Indicador	Meta	Método de evaluación
Cobertura funcional del MVP	>= 90 %	Revisión de requisitos y ejecución de casos de uso
Cumplimiento de reglas críticas	100 %	Pruebas sobre contratos y disponibilidad
Claridad documental	Alta	Comprobación de índice, figuras y estructura
Escalabilidad de la solución	Adecuada	Revisión de arquitectura y

		desacoplamiento
Accesibilidad conceptual	Satisfactoria	Revisión de mockups y criterios de interfaz

12.6. Registro sintético de cambios e incidencias

Elemento	Situación inicial	Evolución / ajuste realizado	Resultado
Arquitectura	Diseño general del sistema	Refuerzo del servicio central GestorAlquiler	Mayor claridad y cohesión
Interfaz	MVP por consola	Definición de mockups móviles futuros	Mejor visión de usabilidad
Base de datos	Modelo inicial básico	Consolidación de relaciones y restricciones	Integridad mejor definida
Documentación	Entregas separadas	Integración en memoria final unificada	Presentación más profesional

13. Bibliografía y webgrafía

Oracle. Java Documentation. Documentación oficial de Java SE 17 para estructuras básicas, orientación a objetos y manejo de fechas.

SQLite Documentation. Documentación oficial del motor SQLite para persistencia ligera en entornos locales.

MySQL Reference Manual. Manual de referencia de MySQL como alternativa de persistencia relacional cliente-servidor.

Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Referencia conceptual para separación de responsabilidades y patrones de diseño.

Sommerville, I. Ingeniería del software. Obra de apoyo para criterios de análisis, planificación, riesgos y calidad.

Material didáctico del ciclo DAM y documentación facilitada en el aula virtual para el Proyecto Intermodular.

14. Conclusión de la memoria

La memoria final de AlquilaSenior presenta un proyecto completo, coherente y bien fundamentado, capaz de responder a una necesidad social real mediante una solución técnica ajustada al nivel y a los objetivos del ciclo DAM. A lo largo del documento se ha demostrado una evolución lógica desde la detección del problema hasta la definición de la arquitectura, la planificación del desarrollo, los criterios de calidad y la visión de futuro del

sistema. Esta coherencia entre análisis, diseño y documentación refuerza la solidez global del trabajo.

El valor del proyecto no se limita a su componente técnico. También destaca por su orientación al usuario, por la sensibilidad hacia la accesibilidad y por el esfuerzo realizado para presentar una propuesta profesional, clara y defendible. La integración de diagramas, mockups, tablas de evaluación, gestión de riesgos y bibliografía convierte esta entrega en una memoria madura y cuidadosamente elaborada. En conjunto, AlquilaSenior refleja una aplicación práctica de los conocimientos adquiridos y una capacidad real para abordar un proyecto software con criterio, orden y visión de mejora.

15. Reflexión personal

La elaboración de este proyecto me ha permitido comprobar que desarrollar software no consiste solo en programar, sino en comprender bien un problema, estructurar una solución viable y justificar cada decisión tomada. Trabajar en AlquilaSenior me ha ayudado a integrar conocimientos de análisis, bases de datos, arquitectura software, diseño orientado a objetos y documentación técnica en un único proyecto con sentido propio. Esa integración ha sido uno de los aprendizajes más valiosos de todo el proceso.

A nivel personal, este trabajo también me ha hecho valorar la importancia de diseñar pensando en las personas y no únicamente en la tecnología. Elegir un proyecto con utilidad social me ha mantenido implicado y me ha permitido enfocar el desarrollo con más motivación y responsabilidad. Considero que el resultado final refleja esfuerzo, evolución y una intención clara de hacer un trabajo serio y de calidad. Por ello, más allá de la calificación, esta memoria representa para mí una demostración de crecimiento académico y personal dentro del ciclo de Desarrollo de Aplicaciones Multiplataforma.